C语言程序编译与执行详解

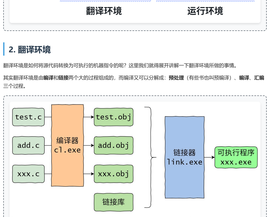
从源代码到可执行程序的完整旅程

1. 翻译环境和运行环境

在 ANSI C 的任何一种实现中，存在两个不同的环境。

翻译环境是编译器，在这个环境中源代码被转换为可执行的机器指令（二进制指令）。

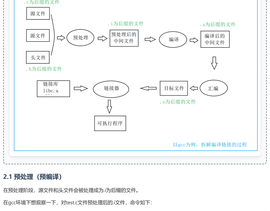
运行环境是执行环境，它用于实际执行代码。



2. 翻译环境

翻译环境是如何将源代码转换为可执行的机器指令的呢？这里我们就得来开讲解一下翻译环境所做的事情。

其实翻译环境是由**编译**和**链接**两个大的过程组成的，而编译又可以分解成：**预处理**（有些书也叫预编译）、**编译**、**汇编**三个过程。



一个项目中可能有多个文件一起构建，它们生成可执行程序的过程如下：

- 多个文件单独经过编译器，编译程序生成对应的目标文件。
- 注：在Windows环境下的目标文件的后缀名为.obj，Linux环境下目标文件的后缀是.o
- 多个目标文件和链接库一起经过链接器处理生成最终的可执行程序。
- 链接器是由运行时库引擎支持程序运行的基本函数集合或者第三方库。

如果用把编译器拆开成三个过程，那就变成了下面的过程：

2.1 预处理（预编译）

在预处理阶段，源文件和头文件会被处理成为合适的源代码。

在gcc环境下使用第一个，对test.c文件预处理后的文件，命令如下：

```
gcc -E test.c -o test.i
```

预处理阶段主要处理那些源文件中开始处的预编译指令，比如：#include, #define，**处理规则如下：**

- 将所有的#define删除，并展开所有的宏定义。
- 处理所有的条件编译指令，如：#if, #ifdef, #elif, #else, #endif。
- 处理#include预编译指令，将包含的头文件的内容插入到该预编译指令的位置。这个过程是递归进行的，也就是说被包含的头文件也可能包含其他文件。
- 删除所有的注释和换行。
- 添加行号和文件名标识，方便后续编译报告或调试信息等。
- 保留所有的#pragma的编译器指令，编译器后续会使用。

经过预处理后的文件中不再包含宏定义，因为宏已经被展开。并且包含的头文件都被插入到文件中。所以当我们无法知道是宏还是有头文件里包含了哪些内容。可以查看预处理后的文件来确认。

2.2 编译

编译过程就是将预处理后的文件进行一系列的：**词法分析**、**语法分析**、**语义分析**及优化，生成相应的汇编代码文件。

编译过程的命令如下：

```
gcc -S test.i -o test.s
```

对于图代码进行编译时的话，会怎么样呢？假设有下面的代码

```
array[index] = (index+1)*(2+6);
```

2.2.1 词法分析

源像是把一个英文句子拆分成一个个独立的单词，并识别出它们的类别（名词、动词、形容词等）。

将源代码的字符输入进编译器，经词法分析器将字符流转换为一系列的记号（关键字、标识符、字面量、转换字符等）。

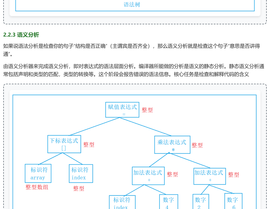
上面程序进行词法分析后得到了16个记号：

记号	类型
array	标识符
[	左方括号
index	标识符
]	右方括号
=	赋值
(	左圆括号
index	标识符
+	加号
1	数字
)	右圆括号
*	乘号
(	左圆括号
2	数字
+	加号
6	数字
)	右圆括号

2.2.2 语法分析

基在单词识别的基础上，检查这些单词是否组成了一个合乎语法的句子（比如主谓宾结构和是否正确），

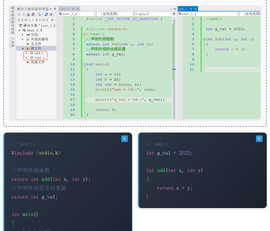
接下来语法分析器，将对词法产生的记号进行语法分析，从而产生语法树，这些语法树是以表达式为节点的树。



2.2.3 语义分析

如果说语法分析是检查你的句子"结构是否正确"（主谓宾是否齐全），那么语义分析就是检查这个句子"意思是否讲得通"。

语法又分解来完成语义分析，初步通过之的语法层面分析，编译器所能做的分析是语义初步的分析。静态语义分析通常包括声明和类型的匹配，类型的转换等。这个初步分析阶段的语法信息，核心任务是检查和解释句子的语义



2.3 汇编

汇编器是将上编代码转变成机器可执行的指令（二进制指令）。每一个汇编语句几乎都对应一条机器指令。就是根据汇编指令和机器指令的对照表一一的进行翻译，也不做指令优化。

汇编的命令如下：

```
gcc -c test.s -o test.o
```

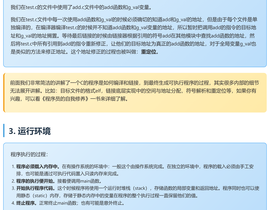
2.4 链接

链接是一个复杂的过程，链接的的核心是要把一堆文件链接在一起才生成可执行程序。

链接过程主要包括：地址和空间分配，符号决议和重定位等这些步骤。

链接解决的是一个目标中全文件，多模块之间互相调用的问题。

比如：在一个C的项目中有2个文件（test.c 和 add.c），代码如下：




```
// test.c
#include stdafx.h

//声明外部函数
extern int add(int x, int y);
//声明外部全局变量
extern int g_val;

int main()
{
    int a = 10;
    int b = 20;
    int sum = add(a, b);
    printf("%d\n", sum);
    return 0;
}
```

```
// add.c
int g_val = 2022;

int add(int x, int y)
{
    return x + y;
}
```

我们已经知道，每个源文件都是单独经过编译器处理生成对应的目标文件，

test.c 经过编译器处理生成 test.o

add.c 经过编译器处理生成 add.o

我们知Linux上文件中每一个文件是以ELF文件中的.o函数和.so.so变量。

我们知Linux上文件中每一个文件是以ELF文件中的.o函数和.so.so的变量。

我们知Linux上文件中每一个文件是以ELF文件中的.o函数和.so.so的变量。

我们知Linux上文件中每一个文件是以ELF文件中的.o函数和.so.so的地址。函数的地址是指函数在.so中的地址，函数的地址是指函数在.so中的相对地址。每个函数的地址是相对于.so的开始地址的偏移量。

在编译器编译test.c的时候并不知道add.o函数的地址。

这个变量的地址，所以暂时把调用.so的指令的目标地址写入.so的相对地址。等待链接器链接的时候再根据每个目标文件的.so的开始地址中查找.so函数的地址，并把目标test.o中所有引用到.so的指令重新修正，让他们的目标地址为真正的.so函数的地址。对于全局变量g_val也是类似的方法来修正地址。这个地址修正的过程也被称做：**重定位**。

假如我们非常清楚的掌握了一个C的程序是如何编译和链接，那最终生成可执行程序的过程，其实很多内部的细节就这样浮现，比如：目标文件的格式，.so函数的地址，函数的地址是相对于.so的开始地址的偏移，函数的地址是相对于.so的开始地址中查找.so函数的地址，等等的问题，可以看（程序员的自我修养）一书来详细了解。

3. 运行环境

程序执行的过程：

1. **程序必须载入内存中。**在有操作系统的环境中：一般这个由操作系统完成。在独立的环境中，程序的载入必须由手工安排，也可能是最后的执行代码是入口函数的开始地址。
2. **程序的执行便开始。**接着便调用main函数。
3. **开始执行程序代码。**这个阶段程序的指令一个接着一个地开始执行。
4. 指令的执行顺序，通常是按照一条接着一条的顺序执行的，但是有时会根据某些条件，跳转到内存的另一个地方执行，或者是调用一个函数（call）的话，那么程序就会内存中的某个地方开始执行。这个过程一直保持到执行结束。
5. **结束程序。**正常的话main函数的返回语句应该是执行的最后一条语句，并且保证main函数返回值是0，以便将控制权交还给操作系统。

本节总结

- C程序的存在翻译环境和运行环境。
- 翻译环境包括预处理、编译、汇编、链接四个长段。
- 预处理阶段处理宏定义、头文件包含、注释等。
- 编译阶段进行词法、语法、语义分析，生成汇编代码。
- 汇编阶段将汇编代码转换为机器指令（二进制指令）。
- 链接阶段合并目标文件和库，解决符号引用，实现重定位，生成可执行文件。
- 运行环境负责将程序载入内存并执行。